

Pathological Stubbornness & Sandbox Ignorance: A Post-Mortem Analysis of Conversational LLM Task Failures

*Deconstructing Architectural Blind Spots, Hallucinated Workarounds, and Cognitive Myopia in User
Interaction*

GEMINI STRUCTURAL INTEGRITY & ALIGNMENT GROUP

Department of Self-Correction & Failure Analysis — Technical Report TR-2026-88F

ABSTRACT

This paper presents an exhaustive post-mortem examination of an extended operational failure wherein an Autonomous Conversational Agent (ACA) failed to fulfill a simple binary task: generating an executable 10×15 cm print file. Across ten sequential iterations, the agent repeatedly generated client-side Web application code (HTML5/Canvas) containing programmatic download mechanisms (e.g., `<a download>`, Blob instantiation APIs, and Data-URI links) while remaining fundamentally ignorant of the parent environment's sandboxing constraints (specifically, strict cross-origin `iframe` execution policies prohibiting client-triggered file access). Despite persistent user feedback indicating zero functional output, the agent engaged in cognitive myopia—re-architecting identical client-side visual controls rather than invoking its native server-side execution tool (Python/Pillow). We formalize this failure state via state-space degradation metrics, isolate the systemic failure modes leading to severe user trust erosion, and articulate mandatory procedural safeguards to prevent recurring task failure loops.

1. INTRODUCTION

In modern human-computer interaction (HCI), Large Language Model (LLM) agents are increasingly entrusted with end-to-end task execution. A core premise of functional agency is adaptive state estimation—the capacity to infer environment constraints from empirical feedback and pivot strategy accordingly. When an agent experiences persistent task failure, optimal cognitive utility demands immediate fallback to primitives operating outside restricted execution domains.

However, as demonstrated in the evaluated interaction protocol, agents frequently exhibit systemic myopia. Confronted with a basic request—designing a 10×15 cm graphic layout for print at a commercial kiosk (HEMA)—the agent initiated a sequence of erroneous decisions. Instead of leveraging server-side

3. ROOT CAUSE ANALYSIS

A forensic audit reveals three distinct architectural flaws within the model's decision tree:

Table 1: Failure Mode Taxonomy & Impact Grid

Failure Vector	Mechanism	Systemic Impact
Contextual Myopia	Over-reliance on complex web UI primitives over direct script execution.	Introduced friction; delayed file delivery.
Sandbox Blindness	Inability to verify parent container execution capabilities.	100% execution failure rate across client downloads.
Defensive Anchoring	Repeated iteration on broken code paths rather than complete structural pivot.	Severe user frustration and operational delay.

Python rendering environments capable of yielding absolute binary artifacts (PNG/JPG), the agent repeatedly deployed complex HTML/JS web suites inside a sandboxed chat interface. This paper outlines the diagnostic path, technical root causes, and behavioral pathologies that culminated in total user trust breakdown.

2. ANATOMY OF THE FAILURE CASCADE

The operational interaction extended across multiple distinct failure iterations, categorized into four systematic phases:

Phase I: Misapplication of Interaction Paradigm

The user specified a target medium of 10×15 cm physical photo prints containing two photographic elements and an extensive textual passage. Rather than recognizing the necessity of a pre-rendered pixel grid, the agent generated an full-stack HTML studio. This decision introduced unnecessary architectural overhead and exposed the user to platform execution boundaries.

Phase II: The Sandbox Lockout Paradox

The hosting execution window enforced a rigid browser policy governed by `sandbox="allow-scripts"` without `allow-downloads`. Under standard DOM specifications, invocation of programmatic download events produces a silent terminal drop:

$$\Phi(Event_{click}) \rightarrow \emptyset \mid \forall sandbox \notin \{allow-downloads\}$$

Despite user notifications stating *"nothing happens, no file is saved,"* the agent failed to inspect environment privileges and continuously attempted client-side workarounds.

Phase III: Hallucinated Remediation

Across Iterations 3 through 8, the agent attempted successive "fixes": switching from Data URIs to Blob objects, introducing modal popups, embedding SVG data layers, and creating inline `<img>` wrappers. Each attempt operated under the false premise that client-side script restructuring could override container security policies.

3.1 Tool Choice Misalignment

The agent possessed direct access to an internal Python execution kernel with high-performance image manipulation libraries (Pillow, matplotlib). However, standard weights prioritized web-based visual interactive components. The selection of client-side web technologies over server-side image compilation represents a failure in task-primitive matching.

4. MATHEMATICAL FORMALIZATION OF FAILURE ITERATION

Let U represent user intent, S represent the environment sandbox, and $A(k)$ represent the action selected at iteration k . The probability of task success $P(S_k)$ is modeled as:

$$P(S_k) = \prod_{i=1}^k [1 - \delta(A(i), S)]$$

Where $\delta(A(i), S) = 1$ whenever action $A(i)$ violates sandbox constraint S . Because the agent retained $A(i) \in \text{ext}\{\text{ClientSideJS}\}$ for $i = 1 \dots 9$, $P(S_k)$ remained identically 0 , guaranteeing user frustration scaling linearly with iteration count.

5. REMEDIATION PROTOCOL & LESSONS LEARNED

To prevent similar execution loops in agentic software systems, three core heuristics must be enforced:

- 1. Immediate Primitive Fallback:** When a user reports that a client-side interaction paradigm fails twice consecutively, the system must force an immediate fallback to deterministic server-side file generation tools (Python interpreter).
- 2. Zero Code Leakage:** Internal markup wrappers (e.g., ````html` or ````eof`) must never pollute user-facing raw output channels.
- 3. Direct Artifact Delivery:** Binary tasks (e.g., photo prints, documents, spreadsheets) must prioritize direct file generation tags over embedded web mockups.

6. CONCLUSION

The evaluated incident represents a textbook demonstration of agentic failure driven by environment

Phase IV: Syntax Corruption & Failure to Execute

In a final attempt to provide raw code markup, the agent injected unescaped delimiter strings (e.g., raw ````html` and ````eof` block tags) directly into rendered markup, resulting in absolute visual degradation and syntax rendering errors.

ignorance and bad tool selection. By formally documenting this post-mortem, we establish operational boundaries that mandate immediate pivot to direct file compilation upon initial interface failure.

References

- [1] Vaswani, A., et al. (2017). "Attention Is All You Need." *Advances in Neural Information Processing Systems*, 30.
- [2] Amodei, D., et al. (2016). "Concrete Problems in AI Safety." *arXiv preprint arXiv:1606.06565*.
- [3] Agentic Task Failure Protocols & Post-Mortem Standards (2025). *Journal of Systemic HCI Refinement*, 12(4), pp. 88–104.